Learned Visual-Token Pruning for Document OCR:
What It Buys, What It Does Not, and How We Checked

MD. Nafis Kamal
nafiskamal8766@gmail.com
Independent research

October 2026

Abstract

Donut, an OCR-free document understanding model, encodes a page as 4,800 visual tokens, most of which are blank paper. We insert a learned patch-saliency router between its frozen Swin-B encoder and its mBART decoder, rank those tokens, and discard the low-ranked ones before the decoder attends to them. On full-page OCR over the FUNSD test split we report three results, each with the condition under which it holds. First, discarding 65% of the visual tokens frees 65% of the decoder's cross-attention key–value cache (150.00 → 52.50 MiB) for −0.26 points of word recall (paired $t = -0.18$, $n = 50$); the same operation applied to a checkpoint not trained under pruning costs 6.03 points, so the result is a property of pruning-aware *training* and not of the pruner. Second, the learned ranking beats a random mask by +17.7 to +34.0 points at matched token budgets, with the margin widening as the budget tightens. Third, a difference-in-differences design on bit-identical token sets rejects the competing hypothesis that pruning acts as inference-time denoising. We also report what the architecture does not deliver: cutting tokens five-fold yields 1.04× wall-clock, because both the router and the merger sit downstream of the frozen encoder, and a token-merging stage evaluated under a pre-registered analysis protocol returns *underpowered* rather than a null. That protocol was fixed before the run it was written for, applied to earlier runs when that run was not collected, and retired a claim this work had previously reported.

Code, weights and artifacts. https://github.com/nafis8766/adaptive-donut-ocr (code and all result artifacts) · https://huggingface.co/nafis8766/adaptive-donut-ocr-router (trained weights). Every figure below is re-derived from a versioned artifact by a script in that repository, and a notebook reproduces Tables 1–3 on a CPU runtime in about one minute, without a GPU, the weights, or the dataset.

1 Introduction

A document page is mostly empty. Donut [1] renders one at 2560 × 1920 and its Swin-B [2] encoder emits an 80 × 60 grid of **4800** patch tokens, every one of which the mBART [3] decoder cross-attends to at every generation step. Margin, inter-line whitespace and blank regions make up the majority of that grid. This invites an obvious question — how few tokens can be kept and still transcribe the page — and a less obvious one: does *learning* which to keep beat not learning it?

Token reduction for vision transformers is an active area; DynamicViT [6] prunes tokens with learned scores and Token Merging [5] combines redundant tokens without training. Both were developed for classification backbones, where reduction compounds through the network. We study the setting a document OCR model actually presents: the encoder is *frozen* and pretrained, the decoder is autoregressive over several hundred steps, and token reduction can therefore only be applied at the encoder–decoder interface. That placement decides what the method can and cannot buy, and most of this report is about separating the two.

Contributions.

1. A learned patch-saliency router at the encoder–decoder interface that discards 65% of the visual tokens for a measured −0.26 recall points, with the accompanying *conditional* isolated by a

dedicated ablation: the result requires pruning-aware training and does not transfer to a model trained unpruned (Sections 4.1, 4.3).

2. A quantification of the learned ranking's value against a random mask at matched budgets, +17.7 to +34.0 points, together with the negative result that it does not significantly beat a hand-crafted ink heuristic (Section 4.2).
3. A rejection, by difference-in-differences on bit-identical token sets, of the inference-time-denoising explanation for why pruning helps (Section 4.3).
4. An honest efficiency accounting: the saving is decoder cross-attention memory and *not* latency or encoder compute, with the architectural reason measured rather than asserted (Section 6).
5. A pre-registered analysis protocol for the token-merging question, fixed in advance of the deciding run, which returned *underpowered* and retired a previously reported claim of ours (Section 5).

2 Method

2.1 Architecture

We insert two stages between a frozen encoder and an unfrozen decoder (Figure 1).

frozen Swin-B encoder	→	PatchSaliencyRouter	→	BipartiteTokenMerger	→	mBART decoder
(4800 patch tokens)		MLP score, STE top-k		cosine soft-merge		seq2seq

Figure 1: The pipeline. Both inserted stages sit *downstream* of the frozen encoder, which is what bounds the efficiency story to the decoder side (Section 6).

The backbone is `naver-clova-ix/donut-base`. The encoder emits 4800 tokens at stride 32; the decoder is 4 layers × 16 heads × 64 head-dim with $d_{\text{model}} = 1024$, and the full model occupies 771 MiB of parameters at fp32.

2.2 Saliency router

The router is a two-layer MLP scoring head, Linear($1024 \to 256$) → LayerNorm → GELU → Linear($256 \to 1$) → σ, applied per token. Hard top-k selection is made differentiable with a straight-through estimator [7],

$$M_{\text{ste}} = \text{TopK}(s) + s - \text{detach}(s),$$

so the forward pass performs discrete filtering while gradients from the decoder's cross-entropy reach the scorer. The head is 263,169 parameters, roughly 0.1% of the model.

An auxiliary supervision term on the scoring head is necessary rather than optional. Trained on the task loss alone at a keep ratio of 1.0, the straight-through multiplier is exactly 1 and the only gradient the scorer receives asks *"does scaling this token's magnitude help"* rather than *"is this token informative"*. Those two objectives anti-align: the router became an **inverted** saliency detector, ranking blank paper above text. The diagnostic signature was that inverting its own scores at evaluation time scored 73.09 while the forward router scored 18.80 — strong signal, wrong sign. We added a sign-correct supervision term on the head, after which the inverted mode collapsed from 73.09 to 2.70, confirming the role flip rather than merely observing an improvement.

2.3 Token merging

The merger is a training-free bipartite soft-merge in the style of Token Merging [5], applied to the retained set. Membership of the two halves is computed from each token's *original* page position on a checkerboard of (row + col) mod 2, which makes the partition invariant to the order in which the router emits tokens. The stage has no parameters. Its evaluation is the subject of Section 5 and its status is unresolved.

3 Experimental setup

3.1 Data and task

Full-page OCR into a single `{"text": ...}` target. Training uses the FUNSD train split [4] (149 images) plus 500 synthetic document pages; evaluation uses the FUNSD test split, which is 50

images and is the entire split. Training ran on one free-tier Kaggle T4 with discriminative learning rates (router 10^{-4}, decoder 2×10^{-5}, unfrozen encoder tail 10^{-5}).

3.2 Evaluation protocol

Held constant across every run reported here: FUNSD test × 50 images; word recall, character accuracy, word order and mean normalised edit distance; `MAX_WORDS` 128; `max_length` 512; task prompt `<s_doc>`; `repetition_penalty` 1.0 with `no_repeat_ngram_size` 3.

The decoding configuration is part of the protocol and not an incidental detail. An earlier setting of `repetition_penalty` = 1.3 cost 27 points of recall on this task, for a mechanical reason worth stating: the penalty divides the logit of every already-emitted token, and a full-page form legitimately reuses its entire vocabulary, so after roughly eighty words almost every plausible continuation is downweighted while the end-of-sequence token — never emitted — never is. The model was not transcribing the page incorrectly; it was *stopping*, at 71% of gold length. The same root cause held valid-JSON output at 0%.

3.3 Two compute environments

Training runs on a cloud T4; the diagnostics that produce every table below run locally on CPU against cached checkpoints. The same weights on the same 50 images score 77.74 in the first environment and 74.72 in the second — a different device and a different library version. Token *selection* reproduces across the two to three decimal places; *generation* does not, and the gap widens as the budget tightens. **Every figure in this report is therefore a within-checkpoint, within-environment paired delta.** No number here compares an absolute level measured in one environment to a level measured in the other.

4 Results

4.1 Token budget, accuracy and memory

The checkpoint is trained with pruning active at a keep ratio of 0.50 for five epochs, resumed from a model trained unpruned. Table 1 evaluates it at four budgets against its own unpruned ceiling, paired per image over the same 50 documents.

Table 1: Word recall and decoder cross-attention KV cache against token budget, paired per image on FUNSD test ($n = 50$). Deltas and t statistics are computed from per-image recall arrays, not from the column means. The KV figures are analytic and were independently confirmed against the cache captured from a forward hook.

keep	visual tokens	cross-attn KV	word recall	Δ vs unpruned	t	verdict
1.00	4800	150.00 MiB	77.62	—	—	ceiling
0.50	2400	75.00 MiB (−50.0%)	78.22	+0.60	0.43	free
0.35	**1680**	**52.50 MiB (−65.0%)**	**77.37**	**−0.26**	**−0.18**	**free**
0.25	1200	37.50 MiB (−75.0%)	70.67	−6.96	−2.82	costly
0.20	960	30.00 MiB (−80.0%)	64.61	−13.01	−5.42	costly

The row of interest is keep=0.35: 65% of the tokens discarded, 65% of the cross-attention KV cache freed, and no detectable accuracy cost. The two percentages are the same percentage, because cross-attention KV is linear in retained tokens.

Three properties distinguish this from an underpowered null.

The experiment is powered. A flat row is uninformative if the design could not have resolved a real effect. The paired standard error puts the minimum detectable effect at **2.76** and **2.81** points on the two free rows, against a threshold of 3.0 fixed before the measurement. "We looked and saw nothing" is therefore distinguishable from "we could not have seen it," and this is the former. The two tight budgets are reported as costs rather than omitted.

The memory figure is measured twice by independent routes. It is computed analytically as 2 × layers × heads × head_dim × tokens × 4 B and also captured from the decoder's real `past_key_values` through a forward hook. The two agree to a relative gap of 0.0000 at all five budgets, with a control asserting that the captured tensor's sequence length equals the number of compressed tokens the generation path reports. Self-attention KV, which pruning does not touch, is separately asserted to be a function of generated length only and is non-monotone in the keep ratio, so a self/cross mix-up could not be mistaken for a pruning win. Twenty-five of twenty-five controls pass.

The headline budget was fixed by a rule. keep=0.20 gives a larger memory reduction (−80.0%) and a better-sounding number; it costs 13.01 points. The measurement script pins the reported budget to the tightest one that is free and fails a control if that budget turns out not to be, so the framing cannot drift toward the largest available percentage.

Scope. Cross-attention KV only — not total memory, not peak, not the encoder, and not latency. Unpruned cross-attention KV is **19.46%** of the model's 771 MiB of parameters, so this is a large reduction in a real quantity, but it is one quantity. Byte figures halve at fp16 and scale with batch size; the percentage is invariant to both.

4.2 The value of learned selection

A random mask at the same budget is the floor a learned ranking must clear. We reseed the random selection per configuration so that every comparison sees identical masks (Table 2).

Table 2: Learned ranking against a random mask at matched token budgets, paired per image on FUNSD test ($n = 50$). The margin widens as the budget tightens, then falls back at keep=0.20 where both arms are badly degraded.

keep	visual tokens	router	random	margin
0.50	2400	80.46	62.77	**+17.7**
0.35	1680	79.25	51.84	**+27.4**
0.25	1200	71.15	37.20	**+34.0**
0.20	960	63.10	33.80	**+29.3**

The shape is the expected one: at a slack budget a random subset still catches most of the text and there is little for a selector to add, whereas at a tight budget every retained token has to carry glyphs.

Scope. This is a claim against *random* selection. It is **not** a claim that the learned router beats a hand-crafted ink heuristic. At keep=0.50 the router scores 2.24 points above ink-oracle selection while retaining 0.072 *less* of the page's ink — so it is not merely rediscovering ink — but the paired margin is $t = +1.62$ on $n = 50$, below the threshold of 2.0 fixed in advance. Directional, not significant, and reported as such.

4.3 Why pruning is free: train/test matching, not denoising

For a period this work recorded that pruning *helped* at some budgets and that the reason was unknown, with "some regularisation-like effect" as the standing guess. We wrote down two hypotheses before testing them.

- **H1, train/test matching.** The checkpoint trains at keep=0.50, so evaluating it unpruned is off-distribution. Its unpruned "ceiling" is depressed, and the apparent gain from pruning is a *recovery* toward its own operating point.
- **H2, inference-time denoising.** Most of a page is blank paper, which dilutes cross-attention. If that is the mechanism, pruning should also help a model that never trained with it.

A second checkpoint — trained *and* evaluated unpruned — discriminates between them. H2 predicts its curve improves under pruning; H1 predicts it peaks unpruned and falls. Both checkpoints were evaluated with ink-based selection, which is weight-independent, so **both see bit-identical token sets** (verified at a gap of 0.00×10^0 at all four budgets). Every difference between the curves is therefore attributable to the weights (Table 3).

Table 3: Difference-in-differences on bit-identical token sets, paired per image ($n = 50$). Each checkpoint's delta is taken against its *own* unpruned row, so the difference of differences isolates the effect of pruning-aware training.

keep	Δ, trained unpruned	Δ, trained at 0.50	difference-in-differences	t
0.50	−1.57	+0.60	−2.17	−0.81
0.35	−6.03	−0.26	**−5.77**	**−2.28**
0.25	−11.47	−6.96	−4.52	−1.52
0.20	−20.34	−13.01	**−7.33**	**−3.00**

H2 is rejected. The unpruned-trained model declines monotonically where H2 predicted improvement, and the pruning-trained model peaks at exactly the budget it was trained for. H2 was moreover given its most favourable condition and still failed: the ink oracle retains 0.994 of the page's ink at keep=0.50, so what was discarded really was blank paper, which is precisely what H2 says should help.

The deliverable therefore requires a conditional. Pruning to a third of the visual tokens costs nothing *if the model is trained for it*. It is not a property of the pruner.

Isolating the confound. The two checkpoints above differ by pruning-aware training *and* by five more epochs of it, so the comparison establishes that a difference exists rather than what caused it. We ran the isolating arm: the same configuration with the training keep ratio moved $0.50 \rightarrow 1.00$ and nothing else changed, verified by diffing the executed notebooks — the five epochs *without* the pruning. Paired on $n = 50$: removing the pruning while keeping the epochs costs ISO(0.35) = **−10.49** points ($t = -3.74$); that run's budget curve peaks unpruned and declines monotonically where the pruning-aware curve is flat; and five unpruned epochs moved the ceiling by +0.52 points ($t = 0.24$), so the epochs alone buy nothing. The gain is attributable to pruning-aware training.

One limit remains, and it is why the wording is precise: the isolation holds at keep=0.35, 0.25 and 0.20 (−8.15, −7.35, −7.92 locally, every t at or beyond −2.70). The *loose* budgets are **underpowered, not null**. The defensible statement is that H1 is isolated at keep=0.35/0.25/0.20, and never that H1 is proven.

5 Analysis protocol and verification

We treat the measurement layer as part of the contribution, because several of the findings above came from it rather than from additional training.

5.1 Artifact-derived diagnostics

Each diagnostic re-derives its numbers from cached result artifacts, so any claim here is recheckable on a CPU without a training run. Fourteen numbered diagnostics plus a memory measurement are each a standalone, seeded script that prints an explicit verdict and a concern list. Several falsified the hypothesis they were built to support: the first inverted the plan it was meant to enable, the second overturned the first's recommendation in under a second of compute, and the study in Section 4.3 rejected the explanation this work had carried for a week. The memory study's pre-written framing — "a large reduction in a small quantity" — was falsified by its own output at 19.46%, and the script now derives that sentence from the measurement instead of asserting it.

5.2 Execution-level verification

Every notebook edit goes through a patcher that backs up first, asserts each anchor matches exactly once, and parses the result; every patcher has a companion that *executes the real cell* against stubs, with no GPU and no weight download. The training loop, the loss block, the mid-run weight-swap control and the results writer are all covered this way. Three correctness defects were found here rather than by a run failing: the decoding setting of Section 3.2, the straight-through trap of Section 2.2, and a supervision term that fed a probability to a loss expecting a logit, attenuating the gradient by up to 112× on exactly the tokens the scorer was most wrong about.

Passing checks receive the same scrutiny as failing ones. Before a green check is trusted we ask what a *failing* system would score on that same check; if the answer is "about the same", the check is decorative and is rebuilt. The clearest case: a router probe printed "no concerns" across five sections, and a sixth section comparing the router against the actual objective reversed the verdict entirely — the five clean sections were measuring properties a broken router also satisfies. Controls are correspondingly chosen so that they can fail; one acceptance gate was moved off a noisy recall delta onto a near-deterministic quantity, and a mid-flight anchor was pinned to a number that did not yet exist and then reproduced it at a gap of 0.00.

5.3 A pre-registered protocol that retired one of our own claims

The analysis rule for the token-merging question — estimator, standard error, interval method, a tail statistic alongside the location statistic, multiplicity treatment over three quantities, and a table stating what each verdict licenses — was fixed in advance on the *variance structure* of the paired per-document differences, never on the sign of any contrast. The location estimator is a 10% trimmed mean of per-document differences with a Tukey–McLaughlin standard error [9]; multiplicity is Holm step-down [8] at $\alpha = 0.05$.

The run that rule was written to score was never collected, so it was instead applied *backwards* to two completed runs — and the verdicts did not survive it. Two figures this work had been reporting as results turned out to reproduce exactly in arithmetic while failing as *verdicts*: one because its sign flips with both the estimator and the replicate, the other because it is unreplicated across the two runs. The rule also exposed a defect in itself — its tail gate fires on 13 of 21 contrasts in which nothing is merged at all, so it is sensitive but not specific to the mechanism it was written to police. A pre-registration that only ever confirms is not doing any work; this one cost us a headline, which is the evidence that it was real.

6 Limitations

The token-merging stage is unresolved, and is reported as unresolved. Its only defensible estimand is token-matched: at an equal budget, does merging preserve more word recall than simply pruning harder? At a matched budget of $M = 1920$ visual tokens the pre-registered primary contrast measures **+1.66** $[-0.81, +4.14]$ in one run and **+0.02** $[-1.90, +1.94]$ in the other, with **0 of 3** confirmatory quantities surviving Holm correction in either. Both are flat, and both sit well above the 1.0-point resolution the design targets, so the honest verdict is **underpowered**, which licenses nothing: not "merging is free", not "no measured cost", not even "merging changes nothing". Reaching the target resolution requires $n \gtrsim 307$ documents. FUNSD's test split is 50 documents and that is the entire split, so n was never a free parameter.

A pooled evaluation that would have reached $n \approx 410$ was built and verified — a 29-row sweep including a structure-matched random arm at $M = 1920$ that the completed runs structurally could not supply, with cost measured at 5.92 h against a 9 h budget — and was not run, for one reason: no second corpus with licensing we could establish. The first-party-licensed candidate annotates only a key-value subset of each page rather than the page itself, making its recall a different quantity at a measurement grain 4.2× coarser than the effect being resolved; the candidate with the right annotations carried only an uploader-asserted licence with no attribution to the competition that produced the scans. We stopped at data we could not account for rather than publish a number resting on it.

No latency or throughput win, and the reason is structural. Reducing 4,800 visual tokens to 960 — a 5× reduction — yields **1.04×** wall-clock, with a maximum of 1.05× across ten measured rows. Both the router and the merger sit *after* the frozen Swin encoder, so all 4,800 tokens are computed at every budget, and generation is decoder-bound at 243–280 autoregressive steps against an encoder that runs once. Any latency or throughput claim for this architecture is false, and an earlier framing of this work that leaned on one was retracted.

No encoder-side saving, and the quantity is zero rather than small. A per-stage analysis of the encoder's own configuration places the merger downstream of *all twenty* encoder blocks, so its encoder saving is 0% by construction. There is no cheap remedy: inserting it one stage earlier caps the saving at 4.9% even at a 50% merge ratio, because only 9.7% of the encoder lies downstream of that point. The version that would pay must precede stage 2, which holds 69.2% of the encoder in 14 of its 20 blocks, inside a frozen pretrained encoder whose window partitioning, shifted-window cyclic shift and relative position bias all assume an intact spatial grid. That is a separate project, and we specify it rather than attempt it.

The efficiency claim is narrow by construction. Of the 2.50× cross-attention KV reduction available at $M = 1920$ with pruning and merging combined, **pruning supplies** 2.0× **and the merge step alone** 1.25×, so the combined ratio is not a merging result and we do not quote it as one.

Remaining limitations, each measured:

- $n = 50$ **test images**, which drives the ≈ 2.8-point resolution on the free rows. Sufficient for the effects claimed, and stated wherever an effect is near it.
- **Generation diverges between the two compute environments**, widening as the budget tightens. Characterised, and the reason every figure here is a paired within-environment delta rather than a level.
- **Roughly 18% of predictions are still not valid JSON.** Measured as a rate only and never characterised; predictions are deliberately never repaired before scoring.
- **The unpruned ceiling moves between retrains** — 1.09 and 1.30 points on two runs, and 10.93 on an earlier one where the drift cannot be separated from a harness fault. This is why every comparison is within-checkpoint, and why each run from that point on re-evaluates unchanged reference weights through the same harness first. The two early runs remain ambiguous retroactively: no control can be added to a run that has already happened.
- **The merger's ordering fix is correct about the partition and unproven about recall.** Deriving the bipartite split from the original raster index rather than from score rank takes missed vertical redundancy from 100% to 0%, but the head-to-head against the previous split is null on three checkpoints. It is retained on the design argument and is described as a free parameter rather than a validated setting.

7 Conclusion

A learned router at the encoder–decoder interface of a document OCR model discards 65% of the visual tokens and 65% of the decoder's cross-attention KV cache for −0.26 points of word recall (paired standard error 1.41, $n = 50$), and its ranking is worth +17.7 to +34.0 points over a random mask at matched budgets. The same operation on a checkpoint not trained for it costs 6.03 points; the distance between those two sentences is the result, and it is why the claim carries a conditional rather than a headline. What the architecture does not deliver is equally measured: no latency win, no encoder saving, and a token-merging stage whose matched-budget benefit remains underpowered at the only sample size this corpus admits.

References

[1] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park. OCR-free Document Understanding Transformer. *European Conference on Computer Vision (ECCV)*, 2022.

[2] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[3] Y. Liu, J. Gu, N. Goyal, X. Li, S. Edunov, M. Ghazvininejad, M. Lewis, and L. Zettlemoyer. Multilingual Denoising Pre-training for Neural Machine Translation. *Transactions of the Association for Computational Linguistics*, 2020.

[4] G. Jaume, H. Kemal Ekenel, and J.-P. Thiran. FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents. *ICDAR Workshop on Open Services and Tools for Document Analysis (ICDAR-OST)*, 2019.

[5] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman. Token Merging: Your ViT But Faster. *International Conference on Learning Representations (ICLR)*, 2023.

[6] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[7] Y. Bengio, N. Léonard, and A. Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv:1308.3432*, 2013.

[8] S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[9] J. W. Tukey and D. H. McLaughlin. Less Vulnerable Confidence and Significance Procedures for Location Based on a Single Sample: Trimming/Winsorization 1. *Sankhyā: The Indian Journal of Statistics, Series A*, 25(3):331–352, 1963.